

Computer Networks

Machine Problem 2:

Designing a Routing Protocol for a Dynamic Network

공지: 2026년 5월 27일 (수)
제출 마감: 2026년 6월 9일 (화) 23:59

1 과제 개요

본 과제에서는 시간에 따라 변화하는 네트워크 위에서 동작하는 라우팅 프로토콜을 설계하고 구현한다. 네트워크는 여러 개의 라우터(노드)와 그들을 잇는 링크로 이루어져 있으며, 시뮬레이션이 진행되는 동안 링크의 cost가 변경되거나, 링크가 끊기거나, 새 링크가 생길 수 있다. 라우터들 사이로 데이터 패킷이 흘러야 하며, 각 라우터는 자기에게 도착한 패킷을 어느 이웃으로 forwarding 할지 스스로 결정해야 한다.

각 라우터는 처음에는 자기 자신과 직접 연결된 이웃의 정보만 알고 있다. 다른 노드까지 가는 경로는 라우터들이 서로 **control message**를 주고받으며 협력해서 알아내야 한다.

네트워크 환경(링크, 토폴로지 변화, 패킷 발생)은 미리 구현되어 **netsim2**라는 시뮬레이터로 제공된다. 본 과제의 목표는 라우터(router)의 동작을 구현하는 것이다. 작성한 동일한 router 코드가 시나리오 안의 모든 노드에서 각각 독립적인 인스턴스로 동작한다 (구체적인 실행 모델은 §3에서 설명한다).

작성한 router는 두 가지 측면에서 평가된다:

- **정확성**: 시뮬레이션 동안 발생한 모든 패킷이 목적지에 도달해야 한다. 단 한 개라도 도달하지 못한 패킷이 있으면 해당 시나리오는 실패로 처리된다.
- **효율성**: 패킷이 도달하기까지 사용한 경로의 총 비용과 라우터들끼리 주고받은 control message의 총 바이트 수를 가능한 한 적게 유지해야 한다.

2 배경 지식

2.1 분산 라우팅 프로토콜

인터넷과 같은 실제 네트워크에서는 어떤 중앙 노드가 전체 토폴로지를 관리하지 않는다. 각 라우터는 자기 주변 정보만 알고 있는 상태에서, 이웃들과 정보를 교환함으로써 멀리 있는 노드까지의 경로를 학습한다. 이런 방식의 라우팅을 **분산 라우팅(distributed routing)**이라고 한다.

본 과제에서 작성하는 라우터도 분산 라우팅 환경에서 동작한다. 따라서 라우터 코드는 **자신의 노드 ID, 전체 노드 수, 직접 연결된 이웃의 ID와 링크 cost**만 가지고 시작한다. 멀리 있는 노드까지의 경로 정보는 이웃과 주고받는 control message를 통해 구축해야 한다.

대표적인 분산 라우팅 알고리즘으로는 강의에서 다룬 **Distance Vector (DV)**와 **Link State (LS)**가 있다. 본 과제에서는 어떤 알고리즘을 선택할지, 어떻게 변형할지, 또는 완전히 새로운 방식을 고안할지를 **자유롭게 결정**한다.

2.2 토폴로지 변화에 대응하기

실제 네트워크에서는 링크가 끊기거나, 새 링크가 생기거나, 링크의 cost가 바뀐다(예: 혼잡으로 인한 metric 변화). 라우터는 이런 변화를 감지하고 라우팅 정보를 업데이트해야 한다. 본 과제에서도 시뮬레이션 도중에 토폴로지가 변하며, 어떤 링크가 변하면 그 **양 끝의 두 라우터에게 notification(콜백)**이 전달된다. 멀리서 일어나는 변화는 직접 알 수 없고, 이웃과의 control message 교환을 통해 간접적으로 알아내야 한다. (본 과제에서는 노드가 추가되거나 사라지는 변화는 없으며, 링크 단위의 변화만 일어난다.)

2.3 라우팅 프로토콜의 효율 측정

라우팅 프로토콜의 효율을 평가할 때는 보통 다음 두 측면을 함께 고려한다.

- **데이터 전달 비용**: 패킷이 src에서 dst까지 가는 데 거치는 경로의 총 link cost (또는 지연 시간). 라우팅 결정이 좋지 않거나 routing table이 stale하면 더 긴 경로를 쓰게 되어 이 값이 커진다.
- **제어 오버헤드**: 라우터들이 routing 정보를 학습/갱신하기 위해 주고받는 control message의 양. 너무 자주, 너무 많이 보내면 대역폭을 낭비하고, 반대로 너무 적게 보내면 routing table이 stale해져 데이터 전달 비용이 커진다.

좋은 라우팅 프로토콜은 이 두 항의 **trade-off**를 시나리오에 맞게 균형있게 다룬다.

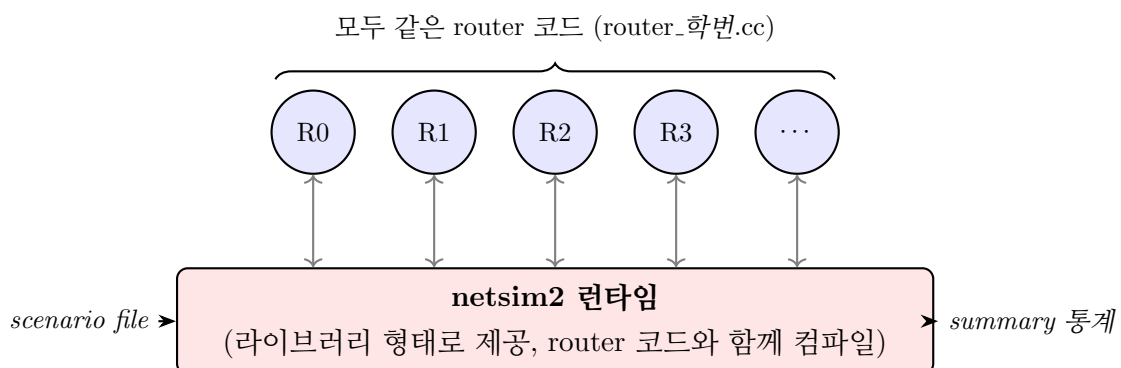
본 과제에서는 위 두 측면을 정량화한 cost 공식으로 라우터를 평가한다 (자세한 정의는 §10):

$$\text{cost} = \text{path_cost_total} + K \times \text{control_bytes_total}$$

3 시스템 구조

3.1 전체 흐름

본 과제의 실행 시스템은 다음과 같은 구조를 갖는다.



3.2 Router (과제에서 구현)

본 과제에서 작성하는 라우터는 C/C++로 된 하나의 소스 파일이며, 시뮬레이터가 동작하는 동안 각 노드마다 한 인스턴스씩 실행된다. 즉, 노드가 10개인 시나리오라면 router 코드는 10번 `router_init()` 콜백을 받아 10개의 독립적인 라우터 인스턴스로 동작한다.

각 인스턴스는 자기 자신의 상태(routing table 등)를 `RouterState` 구조체에 보관하며, 시뮬레이터로부터 이벤트가 발생할 때마다 정해진 콜백 함수가 호출된다. 콜백 안에서 router 코드는 시뮬레이터가 제공하는 함수를 호출하여 이웃에게 control message를 보내거나, 미래 시각에 타이머를 예약할 수 있다.

중요: 라우터 인스턴스들은 서로 독립이다. 한 라우터가 자기 `RouterState`에 저장한 정보는 다른 라우터가 직접 읽을 수 없다. 따라서 전역 변수를 통해 라우터 간 상태를 공유하는 것은 금지된다 (§8 참고).

3.3 netsim2 (시뮬레이션 런타임)

netsim2는 네트워크 환경 전체를 시뮬레이션하는 시뮬레이션 런타임이며, `netsim2.h`와 `netsim2.lib.cc` 두 소스 파일 형태로 제공된다 (별도의 binary 실행 파일은 없다). 작성한 router 코드를 `netsim2.lib.cc`와 함께 컴파일하면, 시뮬레이션 엔진과 router 콜백이 하나로 link된 단일 실행 파일(예: `router.20200001`)이 만들어진다. 이 실행 파일을 실행하면 시뮬레이션이 시작된다 (자세한 컴파일/실행 방법은 §9와 §12를 참고).

netsim2의 동작 순서는 다음과 같다.

1. 시나리오 파일을 읽어 초기 토폴로지를 구성하고, 가상 시계(virtual clock)를 0으로 초기화한다.
2. 각 노드에 대해 `router_init()`을 호출하여 `RouterState`를 생성한다.
3. 시나리오 파일에 정의된 이벤트들(링크 변화, 패킷 발생)을 시각 순서대로 처리한다. router 코드가 호출한 `send_control()`이나 `schedule_wakeup()`에 의해 발생한 후속 이벤트도 적절한 시각에 처리한다.
4. 종료 조건이 만족되면 시뮬레이션을 종료하고, 통계(처리한 패킷 수, 도달한 패킷 수, control byte 합계, cost 등)를 `stderr`에 출력한다.

시뮬레이션 종료 조건: 다음 두 조건이 모두 충족되면 시뮬레이션이 종료된다.

1. 시나리오 파일의 모든 이벤트가 처리되었다.
2. 시나리오에서 발생한 모든 패킷이 처리(delivered 또는 dropped)되었다.

이 시점에 `schedule_wakeup()`으로 예약된 미래 timer나 아직 도달하지 않은 control message가 있더라도 시뮬레이터는 더 이상 콜백을 호출하지 않고 종료한다. 따라서 무한히 timer를 재예약하는 라우터도 무한 루프 없이 종료된다.

4 네트워크 모델

4.1 노드와 링크

네트워크는 N 개의 노드와 그들을 잇는 무방향(undirected) 링크들로 구성된다. 각 노드는 0부터 $N-1$ 까지의 정수 ID를 가진다. 노드 수는 모든 시나리오에서 $1 \leq N \leq 100$ 이다. 시뮬레이션 도중에 노드가 추가되거나 사라지지는 않는다.

각 링크는 양 끝 노드의 ID와 **cost** (1 이상 100 이하의 정수)를 갖는다. 링크는 양방향이며, $u \leftrightarrow v$ 의 cost는 항상 $v \leftrightarrow u$ 의 cost와 같다.

4.2 Link cost = link delay

link cost는 동시에 **link**를 건너는 데 걸리는 시간이기도 하다. 즉, cost가 c 인 링크 위로 message나 packet을 보내면, 도착하기까지 가상 시간으로 정확히 c unit만큼 걸린다. 라우팅 알고리즘이 cost를 최소화하면, 실제로 packet이 빠르게 도달하게 되는 직관적 모델이다.

본 모델에서 link는 **용량 제한이 없다**. 즉, 같은 링크 위로 여러 control message나 packet이 동시에 흐를 수 있으며, 서로 간섭하거나 큐잉(queueing) 지연을 일으키지 않는다. 각 메시지/패킷은 독립적으로 자기 link cost만큼의 시간을 들여 건너간다.

4.3 토폴로지 변화

시뮬레이션 도중에 다음과 같은 토폴로지 변화가 일어날 수 있다.

- **기존 링크의 cost 변경**: 양 끝 노드 양쪽에 `on_link_change(neighbor, new_cost)`가 호출된다.
- **기존 링크의 단절**: 양 끝 노드 양쪽에 `on_link_change(neighbor, NETSIM2_NO_LINK)`가 호출된다. 단절 시점 이후로는 해당 링크로 새로운 `send_control()`이나 `forwarding`을 할 수 없다.
- **새 링크의 생성**: 이전에 연결되지 않았던 두 노드 사이에 새 링크가 생기는 것도 가능하다. 이 경우에도 양쪽 노드에 `on_link_change(new_neighbor, cost)`가 호출된다.

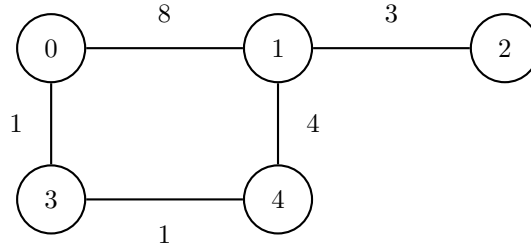
토폴로지 변화는 **자기와 인접한 링크가 영향받을 때만** 알림이 온다. 멀리서 일어난 변화는 직접 알 수 없으며, control message 교환을 통해 알아내야 한다.

변화가 없는 LINK 이벤트: 시나리오의 `LINK u v c` 이벤트가 들어왔는데 그 시점의 현재 cost가 이미 c 와 같다면, 실제 상태 변화가 없으므로 `on_link_change`는 호출되지 않는다. 따라서 router 코드는 콜백이 호출되었다면 링크 상태가 실제로 변했음을 가정해도 된다.

In-flight 메시지 처리: 메시지(control message 또는 packet)가 link로 출발한 시점에 도달 시각이 확정된다. 그 후 link의 cost가 변하거나 link가 down되어도, 이미 출발한 메시지는 원래 예정된 시각에 정상 도달한다. 따라서 `on_control`이나 packet의 도달이 `on_link_change(..., NO_LINK)` 이후에 일어날 수 있다. Link 상태 변화는 **이후의 신규 송신에만** 영향을 미친다.

4.4 토폴로지 예

아래는 5개 노드, 5개 링크로 이루어진 작은 네트워크의 예이다.



이 토폴로지에서 노드 0의 이웃은 1번(cost 8)과 3번(cost 1)이며, 노드 0의 라우터 인스턴스는 `router_init` 시점에 `neighbor_ids = [1, 3]`, `link_costs = [8, 1]`을 받는다. 노드 2, 4와 같이 직접 연결되지 않은 노드까지의 경로는 처음에는 모르며, control message를 주고받으며 학습해야 한다.

5 시간 모델

5.1 가상 시계

`netsim2`는 **discrete-event simulation** 방식으로 동작한다. 시뮬레이터는 가상 시계(virtual clock)를 가지고 있으며, 이 시계는 이벤트가 발생할 때만 앞으로 진행한다. 모든 시각은 정수 단위로 표현된다.

`router` 코드는 `get_now()`를 호출하여 현재 가상 시각을 알아낼 수 있다. 단, 실제 wall-clock 시간이 아니라 시뮬레이션 안의 시각이다.

5.2 이벤트의 종류

가상 시계가 처리하는 이벤트는 다음과 같다.

- **시나리오 이벤트**: 시나리오 파일에 정의된 토폴로지 변화나 패킷 발생.
- **Control message 도착**: `send_control()`로 보낸 메시지가 link cost만큼 후에 이웃에게 도착하는 이벤트.
- **Packet 도착(hop)**: 패킷이 `on_packet()`의 반환값에 따라 다음 노드로 이동하여 도착하는 이벤트.
- **Timer wake-up**: `schedule_wakeup()`으로 예약한 시각에 `on_timer()`가 호출되는 이벤트.

5.3 이벤트 처리 순서

같은 시각에 여러 이벤트가 예약된 경우, 시뮬레이터는 **매번 동일한 순서**로 처리한다. 즉 같은 시나리오 파일을 여러 번 실행해도 결과는 항상 같다. 다만 그 정확한 순서가 어떻게 결정되는지는 구현 detail이므로, `router` 코드가 특정 순서에 의존해서는 안 된다. `router` 콜백은 한 번에 하나씩만 실행되며, 콜백 실행 중에는 가상 시간이 흐르지 않는다 (즉 콜백 시작과 끝의 `get_now()` 값이 동일하다).

6 Packet과 Control Message

6.1 데이터 패킷

데이터 패킷은 시나리오 파일에 정의된 시각에 특정 source 노드에서 발생한다. 각 패킷은 (src, dst) 쌍을 갖는다. 시뮬레이터는 다음과 같이 패킷을 처리한다:

1. 패킷이 src 노드에서 발생한다. 시뮬레이터는 src 노드의 `on_packet(dst)`을 호출한다.
2. router 코드는 다음 hop의 이웃 ID를 반환한다.
3. 시뮬레이터는 해당 링크의 cost만큼 가상 시간을 진행시킨 후, 패킷을 다음 노드로 옮긴다.
4. 다음 노드의 ID가 dst가 아니면 그 노드의 `on_packet(dst)`을 호출한다.
5. dst에 도달하면 시뮬레이터가 자동으로 **delivered**로 기록한다 (`on_packet`이 호출되지 않는다).

경로 비용: 패킷이 거친 모든 링크의 cost 합이 그 패킷의 *path cost*가 된다. Loop가 발생하여 같은 링크를 여러 번 지나면 그만큼 path cost가 증가한다.

TTL: 한 패킷은 최대 $4 \times N$ 번까지만 forwarding될 수 있다 (여기서 N 은 노드 수). 즉, src에서 dst까지 가는 동안 `on_packet()`이 호출되는 횟수의 상한이 $4N$ 이다. 이 횟수를 다 써도 dst에 도달하지 못하면 패킷이 폐기되며, 해당 시나리오가 **FAIL**로 처리된다. 정상 라우팅이라면 이 한계에 닿을 일이 없으며, 보통 forwarding loop가 생긴 경우에만 도달한다.

명시적 drop: `on_packet()`이 -1을 반환하면 패킷이 명시적으로 폐기되며, 이 또한 **FAIL**이다.

src와 dst가 같은 경우: 시나리오에 PKT s s처럼 src와 dst가 같은 이벤트가 포함되어 있으면 시뮬레이터가 path cost 0으로 즉시 delivered 처리하며, `on_packet()`은 호출되지 않는다.

6.2 Control message

Control message는 라우터가 자기의 이웃에게 보내는 byte 배열이다. `send_control()`을 호출하여 임의의 byte payload를 전송할 수 있으며, 그 내용(format, semantics)은 자유롭게 정의한다 (예: distance vector, link state advertisement, hello message 등).

Control message는 다음과 같이 처리된다:

- 호출 시각으로부터 해당 링크의 cost만큼 시간이 지난 후 이웃 노드에게 도달한다.
- 도달 시점에 이웃 노드의 `on_control(from, payload, len)` 콜백이 호출된다.
- Control message의 len 바이트가 cost 계산에 누적된다.
- 한 번에 보낼 수 있는 payload 크기는 **1 이상 65536 이하**의 byte이다 (64 KB까지). 범위를 벗어나면 호출이 무시되고 `stderr`에 경고가 출력된다.

이웃이 아닌 노드에게 보내는 경우: `send_control()`의 neighbor가 현재 이웃이 아니거나 링크가 down 상태이면, 호출은 무시되고 `stderr`에 경고가 출력된다. Cost에는 반영되지 않는다.

Payload 버퍼의 수명: `send_control()`은 호출 시점에 payload 내용을 시뮬레이터 내부 버퍼로 복사한다. 따라서 호출이 반환된 후 payload 버퍼는 자유롭게 해제하거나 재사용해도 된다

(스택에 잡힌 임시 배열을 넘겨도 안전하다). 반대로 `on_control()`이 받는 `payload` 포인터는 콜백이 반환되면 무효화되므로, 콜백 이후에도 데이터가 필요하면 `router` 코드가 직접 복사해 두어야 한다.

7 라이브러리 API

`router` 코드는 `netsim2.h` 헤더에 정의된 함수들을 사용한다. 두 부류로 나뉜다:

- 구현해야 하는 함수 (시뮬레이터가 호출하는 콜백)
- 호출할 수 있는 함수 (시뮬레이터가 제공)

7.1 구현해야 하는 함수

아래 listing의 `#define`과 `struct RouterState`; 선언, 그리고 함수 시그니처는 모두 `netsim2.h`가 제공한다. `router` 코드에서는 `netsim2.h`를 `#include`하기만 하면 되고, 따로 다시 적을 필요는 없다. `router` 코드에서 정의해야 하는 것은 `RouterState`의 멤버(필드)와 각 콜백 함수의 몸체이다.

```
#include "netsim2.h"

// === The following declarations are provided by netsim2.h ===

#define NETSIM2_NO_LINK -1

struct RouterState;    // Student defines the members in their own code

// Called once at router startup. Allocate and return state.
struct RouterState* router_init(int my_id,
                                int num_nodes,
                                const int* neighbor_ids,
                                const int* link_costs,
                                int num_neighbors);

// Called when a directly-connected link changes cost or a new
// link is created. new_cost == NETSIM2_NO_LINK means the link
// went down.
void on_link_change(struct RouterState* s,
                   int neighbor, int new_cost);

// Called when a control message from a directly-connected
// neighbor arrives.
void on_control(struct RouterState* s, int from,
               const uint8_t* payload, int len);

// Called when a packet arrives that is not yet at its dst.
// Return the neighbor ID to forward to. Return -1 to drop (= FAIL).
int on_packet(struct RouterState* s, int dst);

// Called at the virtual time previously requested via
// schedule_wakeup().
```

```
void on_timer(struct RouterState* s);

// Called once at the end of the simulation. Free state if needed.
void router_shutdown(struct RouterState* s);
```

콜백 의미 상세

router_init

시뮬레이션 시작 시점(가상 시각 0)에 모든 노드에 대해 **노드 ID 오름차순**으로 호출된다. `my_id`는 0부터 `num_nodes-1` 사이의 값. `neighbor_ids[i]`와 `link_costs[i]`는 i 번째 이웃의 ID와 그 링크의 cost를 의미하며, i 는 $0..num_neighbors-1$ 이다. `neighbor_ids[]`는 ID 오름차순으로 정렬되어 전달된다. **초기에 이웃이 하나도 없는 격리된 노드**(`num_neighbors == 0`)의 경우 `neighbor_ids`와 `link_costs` 포인터는 NULL일 수 있으니, router 코드는 `num_neighbors`를 먼저 확인해야 한다. 반환된 포인터는 이후 모든 콜백의 `s` 인자로 전달된다.

on_link_change

자기와 인접한 링크의 상태가 변경될 때 호출된다. `router_init`에서 보지 못했던 새 이웃 ID가 들어올 수도 있다 (새 링크 생성). `new_cost == NETSIM2_NO_LINK`이면 그 링크가 더 이상 존재하지 않는다 (down).

on_control

이웃이 `send_control`로 보낸 payload가 도착하면 호출된다. `from`은 보낸 이웃의 ID, `payload`와 `len`은 보낸 그대로의 byte 배열. `payload` 버퍼는 콜백 반환 후 무효화되므로, 필요하면 router 코드가 복사해야 한다.

on_packet

데이터 패킷이 이 노드에 도착했을 때 호출된다 (단 `dst`가 자기 자신이면 호출되지 않음). `dst`는 패킷의 최종 목적지 노드 ID. 반환값은 다음 hop으로 보낼 이웃 노드 ID여야 한다. 여기서 **이웃**이란 **현재 시점에 자기와 link가 up 상태로 연결된 노드**를 뜻한다 (즉 `router_init`에서 받은 초기 이웃 목록을, 그동안 받은 `on_link_change` 알림에 따라 갱신한 결과). 시뮬레이터는 반환된 ID가 이 조건을 만족하는지 검증하며, 만족하지 않으면 `PROTOCOL_ERROR`로 시나리오를 FAIL 처리한다. -1을 반환하면 명시적 drop = FAIL.

on_timer

router 코드가 `schedule_wakeup`으로 예약한 가상 시각에 호출된다. 여러 번 예약된 경우 시각 순서대로 모두 호출된다.

router_shutdown

시뮬레이션 종료 직전에 호출된다. 동적으로 할당한 메모리를 해제하기 위해 사용한다 (선택).

7.2 호출할 수 있는 함수

```
// Send a control message to a directly-connected neighbor.
// If neighbor is not a current neighbor or the link is down,
```

```
// the call is silently ignored (and counts no bytes).
void send_control(int neighbor, const uint8_t* payload, int len);

// Returns the current virtual time.
int64_t get_now(void);

// Schedule on_timer() to fire for the calling router at virtual
// time 'when'. If 'when' <= now, fires at the next event step.
// Multiple scheduled wakeups all fire, in time order.
void schedule_wakeup(int64_t when);
```

호출 가능 시점: 위 함수들은 `router_init`, `on_link_change`, `on_control`, `on_packet`, `on_timer` 중 어떤 콜백 안에서도 호출할 수 있다 (`router_shutdown`에서는 호출하지 말 것).

호출자 추론: 위 함수들은 `my_id`를 인자로 받지 않는다. 시뮬레이터는 현재 어떤 라우터의 콜백을 실행 중인지를 내부적으로 추적하고 있으며, 그 라우터의 동작으로 처리한다.

8 Router 구현 요구사항

8.1 기본 규칙

1. `netsim2.h`를 `#include`해야 한다.
2. 위 §7에 나열된 콜백 함수를 모두 구현해야 한다 (몸체가 비어 있어도 정의는 있어야 한다).
3. 라우터 코드는 `stdout`과 `stdin`을 사용하지 말 것 (채점 환경에서 다른 출력과 충돌할 수 있음). 디버깅용 출력은 `stderr`로 보내면 채점에 영향을 주지 않는다 (시뮬레이터의 통계도 `stderr`로 출력되므로 두 출력이 섞일 수 있음을 유의).
4. 표준 C/C++ 라이브러리 외 외부 라이브러리 사용 금지.
5. 단일 `thread`로만 동작해야 한다. `std::thread`, `pthread.*`, `fork`, `exec`, signal handler, socket/pipe 등 OS 수준의 동시성/IPC 메커니즘은 금지된다.
6. 같은 router 코드를 같은 시나리오에 두 번 실행하면 항상 동일한 결과(status, cost)가 나와야 한다 (채점 재현성을 위함). 따라서 router 코드는 `rand()` (고정 seed로 `srand` 한 경우는 제외), `time(NULL)`, `/dev/urandom` 등 외부 random 소스에 의존하지 말 것.

8.2 분산성 규칙

각 라우터는 자기 `RouterState`에 보관된 정보와 콜백을 통해 받은 정보만 사용해야 한다.

- 전역 변수나 `static` 변수로 라우터 간에 상태를 공유해서는 안 된다 (단, read-only 상수나 상태가 없는 헬퍼 함수는 무방하다).
- 시뮬레이터 내부 자료구조에 접근하거나, 시나리오 파일이나 다른 파일을 직접 읽으려고 시도해서는 안 된다.
- 다른 라우터의 `RouterState`를 임의로 참조해서는 안 된다.

위 규칙을 위반한 코드는 채점 시 **0점** 처리될 수 있다.

8.3 Forwarding 규칙

- on_packet이 반환하는 다음 hop은 반드시 현재 이웃이며 링크가 up 상태여야 한다.
- Loop를 만들지 않도록 주의할 것 (TTL 초과 시 FAIL).

9 netsim2 사용법

9.1 명령줄

router 코드를 netsim2_lib.cc와 함께 컴파일하면 단일 실행 파일이 만들어진다 (예: router_20200001). 그 실행 파일은 다음과 같이 실행한다:

```
./router_20200001 --scenario FILE
```

9.2 인자 설명

--scenario FILE

시나리오 파일 경로.

9.3 시나리오 파일 포맷

시나리오 파일은 토폴로지, 초기 이웃 관계, 그리고 시뮬레이션 동안 발생할 이벤트들을 담고 있는 텍스트 파일이다. 파일은 NODES와 EDGE 라인(토폴로지)이 먼저 나오고, 그 다음 T 라인(이벤트)이 나온다.

같은 시각(T의 값이 같은)에 여러 이벤트가 있는 경우, 시뮬레이터는 **파일에 등장한 순서대로** 처리한다.

시뮬레이터가 제공하는 시나리오 파일은 다음을 만족함이 보장된다:

- Self-loop가 없다 (EDGE u u c 또는 T t LINK u u c 형태의 라인은 등장하지 않는다).
- 같은 두 노드 사이에 중복된 EDGE 라인이 없다 (한 토폴로지에 최대 하나의 링크).
- T의 값은 비내림차순(non-decreasing)이다.
- 모든 EDGE/LINK/PKT 라인의 노드 ID는 0부터 N-1 사이의 유효한 값이다.
- EDGE 라인의 cost는 1 이상 100 이하의 정수이다. LINK 이벤트의 cost는 1 이상 100 이하의 정수이거나, 단절을 의미하는 -1이다.

```
# Lines starting with '#' are comments. Blank lines are ignored.

# === Topology ===
# Number of nodes
NODES <N>

# Initial links (undirected, cost in [1, 100])
EDGE <u> <v> <cost>
EDGE <u> <v> <cost>
```

```

...

# === Events ===
# Each event line uses the format below.
# Virtual time t is a non-negative integer.
# t must be non-decreasing across event lines.

T <t> LINK <u> <v> <cost>      # at time t, set link u-v cost
T <t> LINK <u> <v> -1          # at time t, bring link u-v down
T <t> PKT  <src> <dst>          # at time t, a packet appears at src
                                # destined for dst

```

9.4 시나리오 파일 예

```

NODES 5

EDGE 0 1 8
EDGE 1 2 3
EDGE 1 4 4
EDGE 3 0 1
EDGE 3 4 1

T 100  PKT  1 0
T 200  PKT  2 4
T 500  LINK 1 3 1
T 800  PKT  0 4
T 1200 LINK 1 3 -1
T 1500 PKT  2 0

```

9.5 출력 형식

시뮬레이션이 끝나면 netsim2는 stderr에 다음과 같은 통계를 출력한다:

```

=== netsim2 summary ===
status: SUCCESS
num_nodes: 5
sim_time: 1509
packets_total: 4
packets_delivered: 4
packets_dropped: 0
path_cost_total: 24
control_bytes_total: 312
K: 10
cost: 3144

```

status 값:

- SUCCESS: 모든 패킷이 정상 delivered됨.
- FAIL_DROPPED: TTL 초과 또는 명시적 drop으로 패킷이 도달하지 못함.

- `PROTOCOL_ERROR`: `on_packet`이 잘못된 `next hop`을 반환하는 등 규칙 위반.

$$\text{cost} = \text{path_cost_total} + K \times \text{control_bytes_total}$$

10 Cost 정의 및 채점 방법

10.1 Cost 공식

$$\text{cost} = \text{path_cost_total} + K \times \text{control_bytes_total}$$

`path_cost_total` 모든 delivered 패킷이 거친 링크 `cost`의 합. Loop를 돌면 그만큼 비용이 증가.

`control_bytes_total` 모든 라우터가 `send_control()` 호출을 통해 link로 출발시킨 byte 수의 합. 호출이 무시된 경우(이웃이 아닌 노드, len 범위 밖 등)는 byte로 집계되지 않는다.

`K` 본 과제의 모든 시나리오에서 $K = 10$ 으로 고정.

`cost`는 작을수록 효율적이다.

10.2 시나리오별 점수

채점은 여러 개의 **비공개 시나리오**(hidden test set)에서 제출된 router를 실행하여 진행된다. 각 시나리오는 100점 만점이다.

- `status`가 `SUCCESS`가 아니면 (즉 패킷이 1개라도 drop되거나 `PROTOCOL_ERROR`가 발생하면) **0점 (FAIL)**.
- `PASS`인 경우 일정 수준의 **기본 점수**가 부여되며, 그 이상은 `cost`에 따라 차등 부여된다. `cost`가 낮을수록 높은 점수를 받으며, `PASS`한 제출물 중 `cost`가 가장 낮은 **상위 10%**는 모두 **100점**을 받는다.

10.3 최종 점수

전체 시나리오 점수의 **평균**이 최종 점수가 된다. 한 시나리오에서 실패해도 다른 시나리오에서 점수를 받을 수 있다.

10.4 시나리오 다양성

비공개 시나리오들은 다음과 같은 축에서 다양하게 구성된다 (정확한 파라미터는 채점 시점까지 비공개):

- 노드 수 N (작은 네트워크부터 큰 네트워크까지).
- 토폴로지 형태 (sparse / dense, ring / mesh / tree 등).
- 토폴로지 변화 빈도 (안정적인 네트워크부터 자주 변하는 네트워크까지).

- 트래픽 패턴 (드문 패킷, 집중적인 패킷, 특정 노드로 몰리는 패킷 등).
- 시뮬레이션 길이.

10.5 시나리오의 보장 사항

모든 (validation 및 hidden test) 시나리오는 다음을 만족함이 보장된다:

- PKT src dst 이벤트가 발생하는 시점에 현재 up 상태인 링크들만으로 src에서 dst로 가는 경로가 **최소 하나는 존재**한다. 즉, partition으로 인해 본질적으로 도달 불가능한 패킷은 발생하지 않는다.
- PKT 이벤트가 발생한 후, 그 패킷이 dst에 도달할 만큼 충분한 시간 동안 **src와 dst 사이에는 어떤 경로든 항상 적어도 하나가 유지**된다. 따라서 router가 일단 forwarding을 시작한 패킷이 “경로 자체가 사라져서” 도달 불가능해지는 일은 없다 (잘못된 forwarding 결정으로 loop를 만들거나 dead end로 보내는 일은 구현 책임).
- 시나리오 이벤트의 수와 시뮬레이션 길이는 유한하다.

router가 그 경로를 발견하느냐는 구현의 몫이다. 시나리오는 경로가 존재함을 보장할 뿐이다.

11 Validation 시나리오

다음 시나리오들은 작성한 router의 동작을 검증할 수 있도록 **공개**되며, 채점에는 사용되지 않는다. 시나리오 파일은 과제 패키지과 함께 제공된다.

- **Validation 1** (val1.scn) – 작은 네트워크, 안정적, 적은 트래픽
- **Validation 2** (val2.scn) – 중간 크기, 변화 있음, 적당한 트래픽
- **Validation 3** (val3.scn) – 큰 네트워크, 변화 빈번, 많은 트래픽
- **Validation 4** (val4.scn) – 토폴로지 단절(partition) 후 재연결이 있는 시나리오 (단절 기간 동안에는 cluster 내부 PKT만 발생; §10의 보장에 따름)

실행 예시 (router_20200001은 컴파일된 router 실행 파일):

```
./router_20200001 --scenario val1.scn
./router_20200001 --scenario val2.scn
./router_20200001 --scenario val3.scn
./router_20200001 --scenario val4.scn
```

실제 *hidden test set*은 위 시나리오들과 비슷한 구조를 따르되, 다른 토폴로지, 다른 이벤트 패턴, 다른 노드 수로 구성된다.

12 제출 방법

12.1 제출 파일

본인이 작성한 router 코드 파일 하나만 제출한다. 파일명은 router_<학번>.cc 형식으로 한다 (예: router_20200001.cc).

netsim2.h, netsim2_lib.cc 등 제공된 파일은 함께 제출하지 않는다.

12.2 컴파일

채점 스크립트는 제출된 router 코드를 netsim2.h, netsim2_lib.cc와 같은 디렉토리에 두고 다음과 같이 컴파일한다:

```
g++ -O2 -o router_20200001 router_20200001.cc netsim2_lib.cc
```

컴파일이 실패하거나 위 옵션 외 추가 라이브러리가 필요하면 0점 처리한다. 코드 내용은 C 스타일이든 C++ 스타일이든 자유이며, g++로 컴파일 가능하기만 하면 된다.

12.3 제공 파일

다음 파일이 함께 제공된다:

- netsim2.h – API 헤더 (router 코드에서 #include)
- netsim2_lib.cc – 시뮬레이션 런타임 구현 (router 코드와 함께 컴파일)
- val1.scn ~ val4.scn – validation 시나리오

netsim2라는 이름의 별도 실행 파일은 제공되지 않는다. netsim2_lib.cc가 main()과 시뮬레이션 엔진을 모두 포함하고 있어, router 코드와 함께 link되어 단일 실행 파일을 만든다.

12.4 환경

- 채점 환경: cspro5 서버 (Linux x86_64)
- 컴파일러: g++ (GCC)
- 표준 C/C++ 라이브러리만 사용 가능 (외부 라이브러리 금지)

12.5 지각 제출

마감일 이후 최대 3일까지 제출 가능. 하루마다 10% 감점.

13 표절 정책

- 아이디어와 알고리즘은 친구와 토론 가능하다.
- 그러나 코드를 공유하거나 그대로 베끼는 것은 금지한다 (수강생 사이뿐 아니라 인터넷에 있는 코드도 마찬가지).

- AI 도구(ChatGPT, Claude, Copilot 등)를 사용하여 코드를 작성하는 것은 **허용**한다. 단, AI가 만든 코드를 무비판적으로 제출하지 말고, 본인이 **이해하고 검증**해야 한다.
- 표절이 발견되면 **0점** 처리되며, 학칙에 따라 추가 처벌을 받을 수 있다.